스터디 트래커의 이벤트 기반 설계 적용과 회고

1. 문제 인식

스터디 트래커 1.0 은 스케줄러 기반의 동기 흐름으로 설계되었다.

Slack 전송, 요약 생성, DB 저장이 하나의 절차적 흐름에 결합되어 있었으며,

요약 실패 시 Redis 큐를 활용한 재시도 구조만 부분적으로 비동기 처리를 도입한 상태였다.

다음과 같은 한계를 인식했다.

- 실패 유저 단위 추적이 어렵다
- 책임이 하나의 메서드에 강하게 결합되어 있다
- 병렬 처리 확장이 어렵다
- 장애 발생 시 전파 가능성이 존재한다

또한 부하 테스트를 통해 스케줄러 중심 구조에서 처리 지연 가능성을 확인했고,

이를 계기로 이벤트 기반 구조를 실험적으로 검토하게 되었다.

2. Redis 기반 이벤트 처리 실험

현재 시스템은 전체 EDA 구조는 아니지만,
요약 실패 시 Redis 큐를 활용한 Producer–Consumer 패턴을 부분적으로 적용했다.

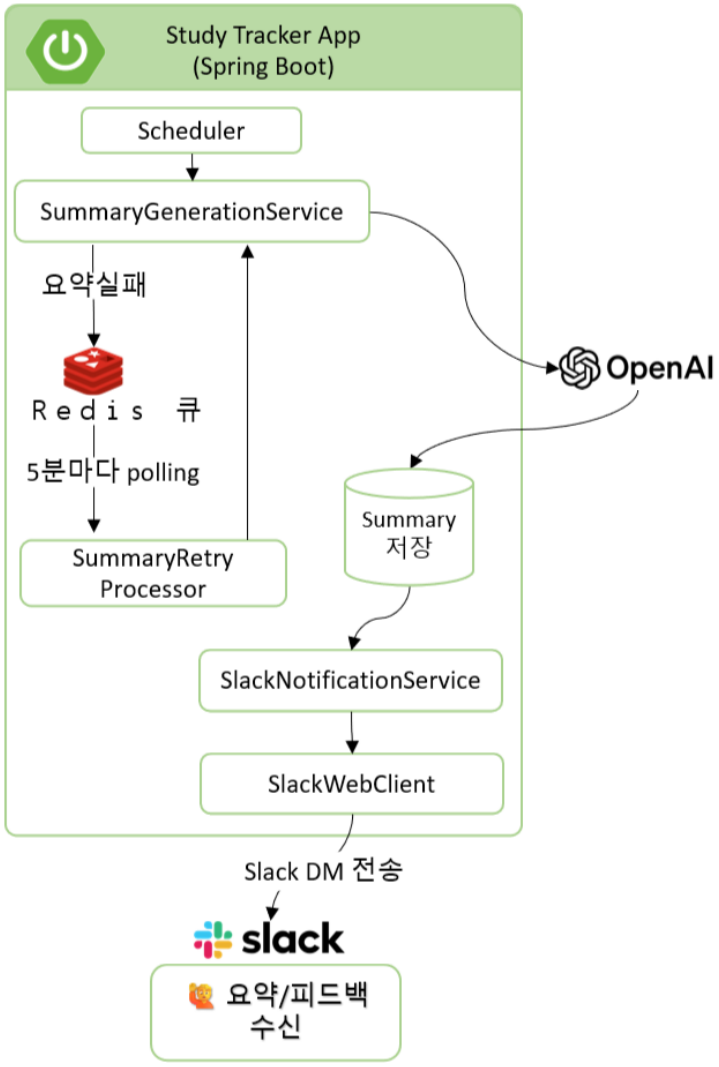
구현 방식

- 실패 요청을 summary-retry-queue 에 저장

- 별도 RetryProcessor 가 polling 하여 재처리
- 최대 4 회 재시도 후 로그 기록

실험을 통해 확인한 점

- at-least-once 처리 보장 가능
- 실패 요청 유실 방지 가능
- 그러나 polling 기반 구조는 병렬 확장에 한계 존재



3. Spring Event 기반 대안 설계 검토

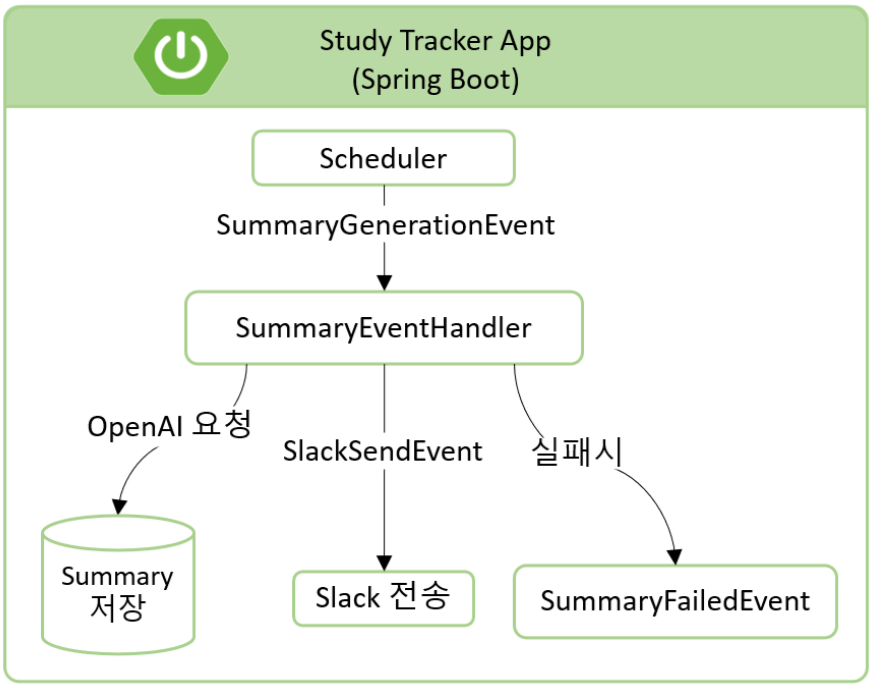
로컬 이벤트 기반 구조(Spring ApplicationEventPublisher)를 검토했다.

기대 효과

- 책임 단위 분리
- 이벤트 체이닝을 통한 확장성 확보
- 후속 처리 로직 삽입 용이

한계

- JVM 메모리 기반 이벤트로 영속성 없음
- 애플리케이션 종료 시 이벤트 유실 가능
- 흐름 추적 난이도 증가



4. 설계 인사이트

이 실험을 통해 다음을 인식했다.

- 이벤트 기반 아키텍처는 확장성과 SRP 측면에서 유리하다.
- 그러나 복잡도 증가와 관찰성 저하라는 비용이 따른다.
- 작은 규모의 단일 책임 시스템에서는 절차적 설계가 오히려 실용적일 수 있다.
- 메시지 브로커 도입은 “기술 트렌드”가 아니라 “트래픽과 조직 규모”를 기준으로 결정되어야 한다.